

Universidad Nacional de Trujillo

Facultad de Ciencias Físicas y Matemáticas

Ingeniería Informática



Base de Datos Avanzada

Temas.





Universidad Nacional de Trujillo



Introducción

Introducción.

La integración de la minería de datos y las bases de datos multimedia en entornos Big Data permite analizar grandes volúmenes de información heterogénea, como imágenes, audio, video y texto, de manera eficiente y escalable.

A través de frameworks especializados, se facilita el procesamiento distribuido, la extracción de patrones y la generación de conocimiento útil a partir de datos complejos.

Esta convergencia tecnológica resulta clave en aplicaciones como reconocimiento de imágenes, análisis de redes sociales y sistemas inteligentes, donde la diversidad y el volumen de datos requieren soluciones robustas y de alto rendimiento.



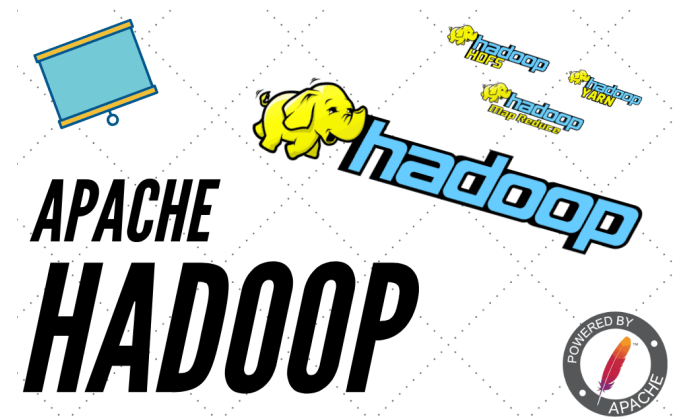
Universidad Nacional de Trujillo



Desarrollo

Apache Hadoop

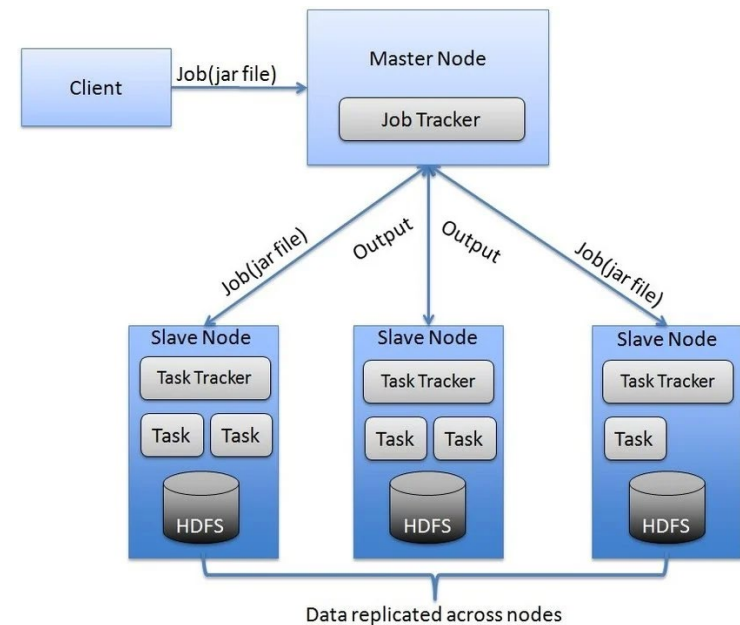
Apache Hadoop es un marco de software de código abierto diseñado para el procesamiento y almacenamiento distribuido de grandes volúmenes de datos. Desarrollado originalmente por Doug Cutting e inspirado en los trabajos de Google, se convirtió en un pilar del ecosistema de big data moderno al permitir el análisis escalable en clusters de hardware estándar.



Arquitectura y componentes

Hadoop se basa en una arquitectura distribuida de maestro-trabajador. Su sistema de archivos, divide y replica los datos en múltiples nodos para asegurar tolerancia a fallos y acceso paralelo.

MapReduce gestiona el procesamiento paralelo, mientras que YARN (Yet Another Resource Negotiator) coordina los recursos del clúster. El módulo Common proporciona utilidades compartidas por todo el sistema.

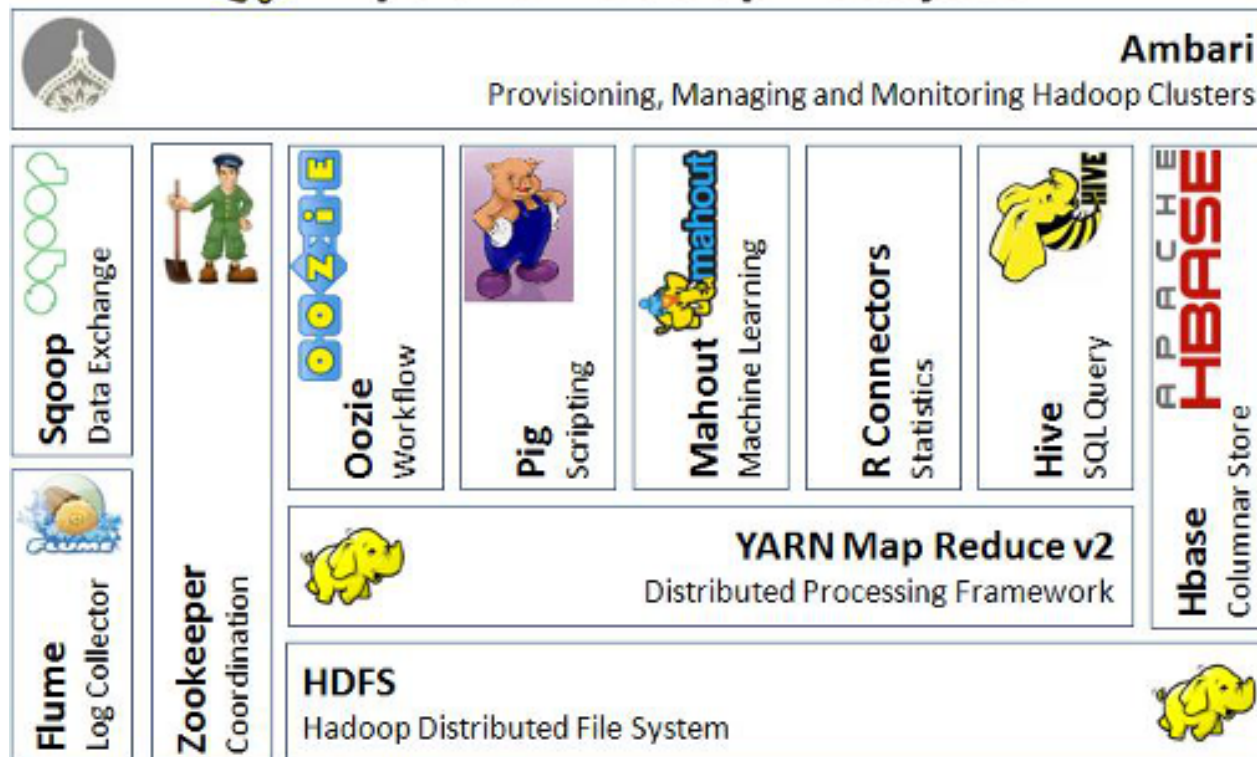


Ecosistema y extensiones

El proyecto dio origen a un amplio ecosistema de herramientas complementarias. Entre ellas destacan Apache Hive, Apache Pig, Apache HBase y Apache Spark, que se integran para ofrecer almacenamiento, análisis, y aprendizaje automático sobre grandes volúmenes de datos.



Apache Hadoop Ecosystem



Apache Spark

Apache Spark, es un motor de procesamiento distribuido de código abierto desarrollado por la Apache Software Foundation, para análisis y manipulación de grandes volúmenes de datos. Está diseñado para ejecutar cargas de trabajo de Big Data a gran velocidad y con tolerancia a fallos, tanto en procesamiento por lotes como en tiempo real, y se integra con diversos lenguajes y plataformas.

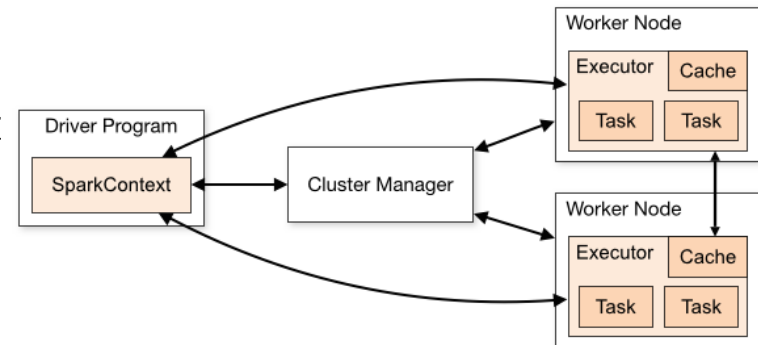


Arquitectura y funcionamiento

Spark, utiliza una arquitectura maestro-trabajador donde el Spark Driver coordina un clúster de nodos de trabajo a través de gestores como YARN, Kubernetes o Mesos.

Su estructura básica son los Resilient Distributed Datasets (RDD), colecciones distribuidas tolerantes a fallos que permiten operaciones paralelas en memoria.

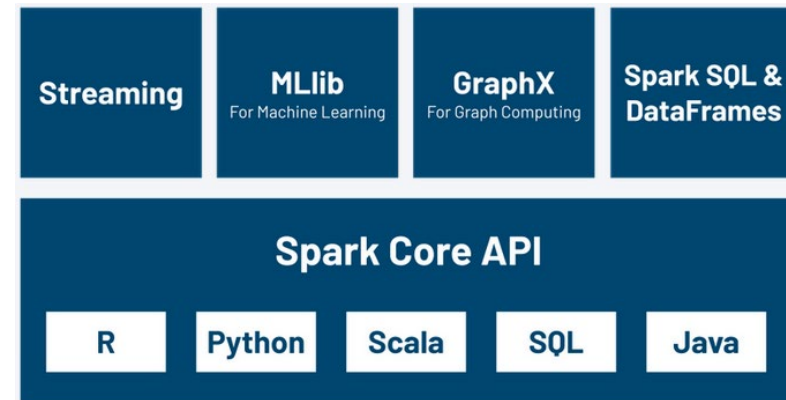
El planificador DAG (Directed Acyclic Graph) optimiza la ejecución de tareas y permite la recuperación ante fallos sin reiniciar todo el proceso.



Ecosistema y biblioteca

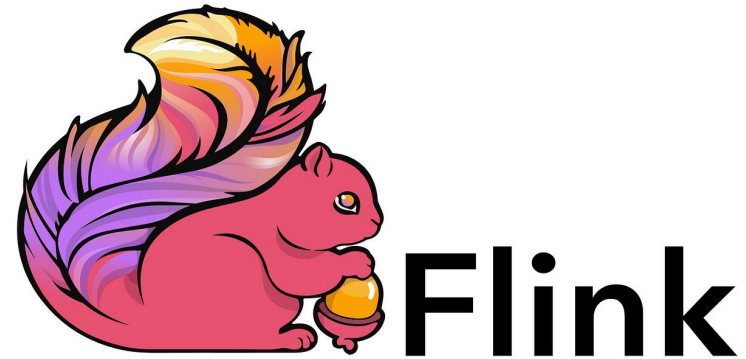
Sobre su núcleo (Spark Core) se construye un conjunto de bibliotecas integradas:

- ✓ Spark SQL y DataFrames para consultas estructuradas y compatibilidad SQL.
- ✓ Spark Streaming para análisis en tiempo real de flujos de datos.
- ✓ MLlib para machine learning (escalado, clasificación, regresión, etc.).
- ✓ GraphX para procesamiento de gráficos y redes sociales.



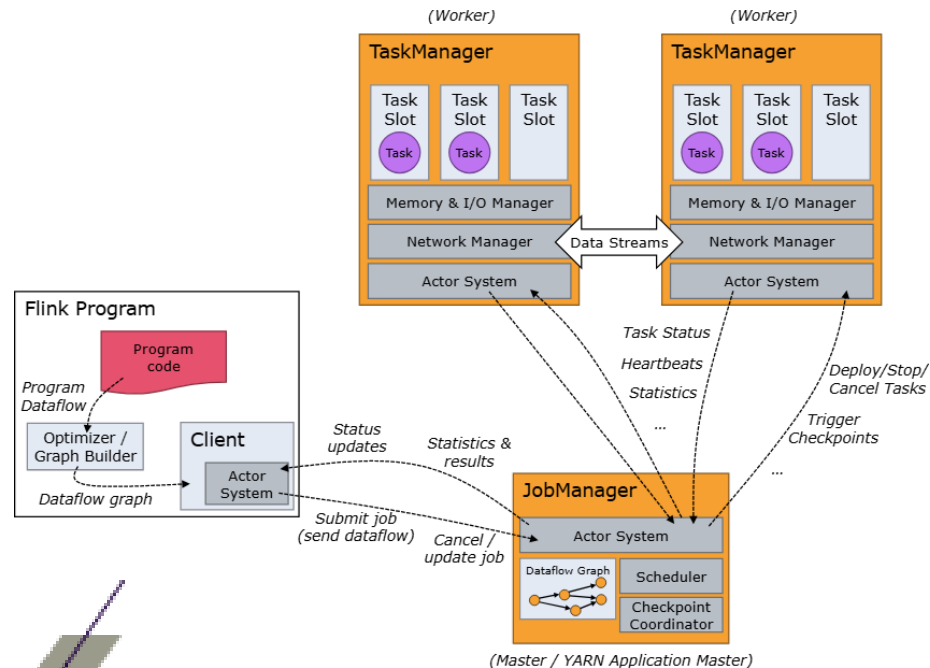
Apache Flik

Apache Flink, es un marco de procesamiento distribuido de código abierto diseñado para realizar cálculos con estado sobre flujos de datos ilimitados (streaming) y conjuntos limitados (batch). Desarrollado bajo la The Apache Software Foundation, ofrece una arquitectura de baja latencia y alto rendimiento que impulsa aplicaciones de análisis en tiempo real, ETL continuo y sistemas impulsados por eventos. Su versión estable más reciente es Apache Flink 2.2.0 (diciembre de 2025)



Arquitectura y Funcionamiento

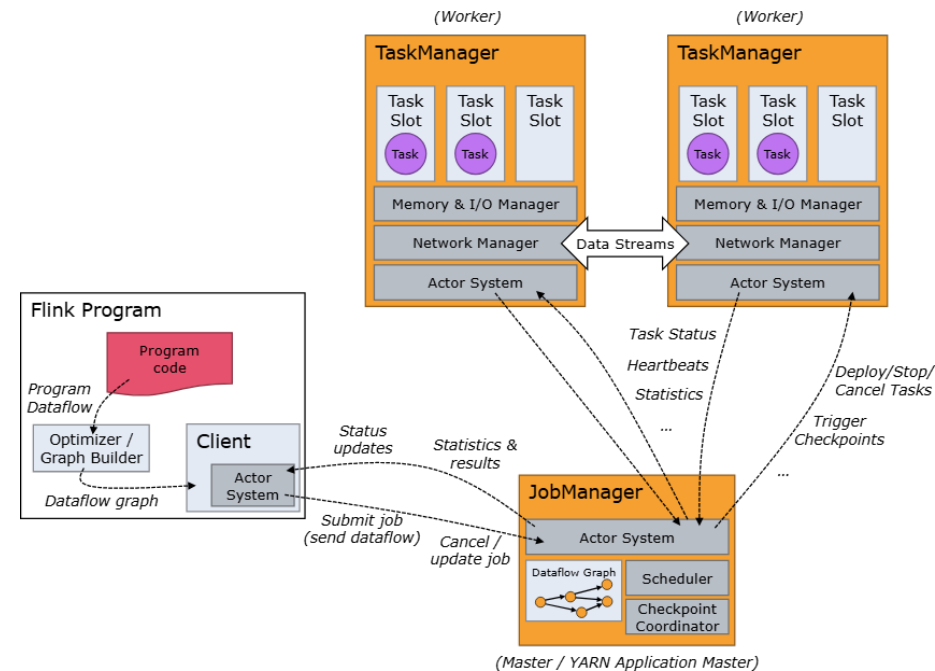
Apache Flink, implementa un motor de flujo de datos acíclico compuesto por fuentes, transformaciones y sumideros. Gestiona el estado de las aplicaciones mediante puntos de control (checkpoints) automáticos y puntos de guardado (savepoints), lo que permite reinicios seguros sin pérdida de datos. Su modelo de “tiempo de evento” ofrece tolerancia a datos desordenados y manejo sofisticado de datos tardíos



Apis y ecosistemas

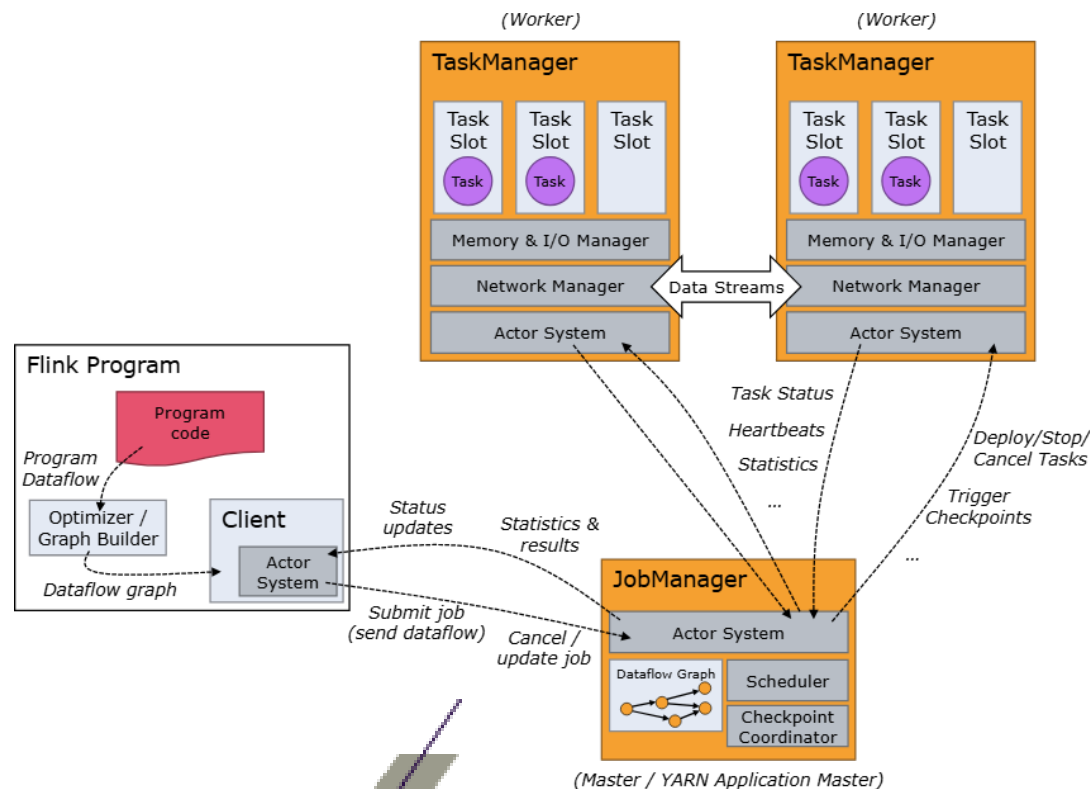
Proporciona varias capas de abstracción:

- ✓ **API SQL y Table:** para consultas declarativas en datos de streaming y batch.
- ✓ **API DataStream:** para operaciones de ventana, agregaciones y detección de patrones.
- ✓ **ProcessFunction:** control detallado del estado y del tiempo de procesamiento.



Apis y ecosistemas

El ecosistema incluye bibliotecas para aprendizaje automático (Flink ML), funciones con estado, y operadores para Kubernetes y servicios de nube. También cuenta con conectores a sistemas como Apache Kafka, Amazon Kinesis, HBase, Elasticsearch y JDBC



Apache kafka

En la actualidad, Apache Kafka constituye una tecnología clave para la gestión de flujos de datos en tiempo real dentro de las organizaciones. Su adopción se ha extendido ampliamente entre las grandes corporaciones, estimándose que más del 80 % de las empresas Fortune 500 emplean Kafka para implementar canalizaciones de datos escalables, integrar aplicaciones heterogéneas y habilitar el análisis de información en tiempo real dentro de arquitecturas orientadas a eventos.



Apache kafka

Apache Kafka se ha consolidado como uno de los estándares de facto para el procesamiento de eventos en tiempo real. De acuerdo con la documentación oficial del proyecto, más del 80 % de las **empresas Fortune 100** lo utilizan para implementar canalizaciones de datos de alta disponibilidad, análisis en tiempo real, integración de sistemas y arquitecturas orientadas a eventos.

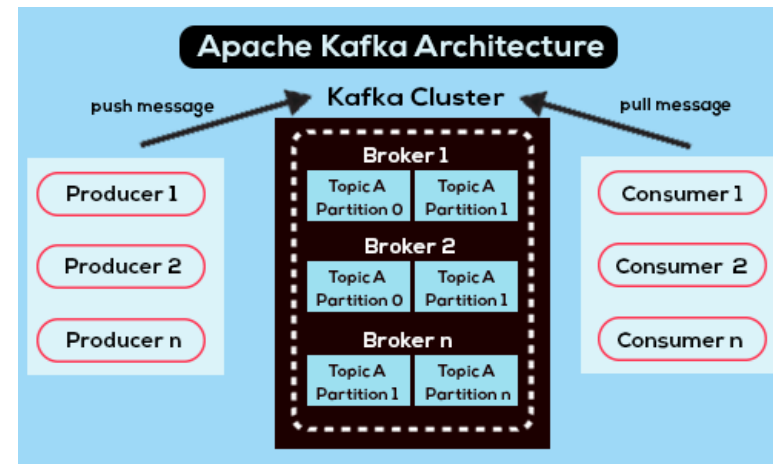


Arquitectura

Kafka se basa en un modelo de publicación/suscripción distribuido. Los productores publican mensajes en temas, que se dividen en particiones distribuidas entre varios brokers (servidores) dentro de un clúster.

Esta estructura garantiza durabilidad mediante replicación y persistencia en disco.

Kafka puede procesar millones de eventos por segundo con latencias de milisegundos, escalando de forma horizontal al añadir más brokers.



Apache storm

Apache Storm, es un marco de cómputo distribuido de código abierto diseñado para el procesamiento en tiempo real de flujos de datos. Permite analizar y responder instantáneamente a grandes volúmenes de información, siendo ampliamente utilizado en sistemas de analítica, monitoreo y procesamiento continuo de eventos. La arquitectura permite una alta tolerancia a fallos y escalabilidad horizontal.



Apache Samza

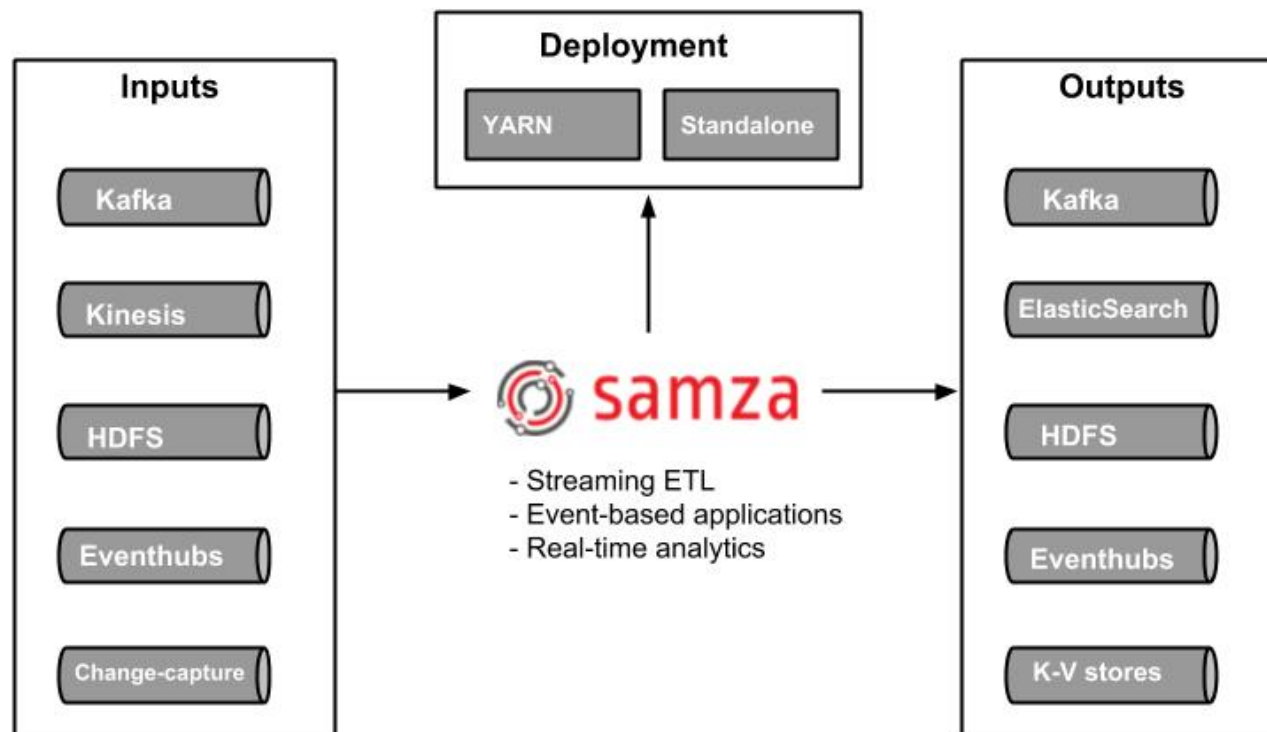
Esta diseñado para procesar grandes volúmenes de información en tiempo real con alta escalabilidad, tolerancia a fallos y baja latencia.

Se ampara también en el uso de Kubernetes, es un sistema de orquestación de contenedores de código abierto que automatiza el despliegue, la administración, el escalado y la operación de aplicaciones distribuidas, optimizando el uso de recursos y garantizando alta disponibilidad mediante mecanismos de balanceo de carga, autorrecuperación (self-healing) y escalado automático.



Apache Samza

Su arquitectura se integra de forma nativa con sistemas de mensajería distribuida como Apache Kafka, que actúa como fuente y almacenamiento de los flujos de eventos, y utiliza gestores de recursos como Apache YARN.



Arquitectura y funcionamiento

Apache Samza utiliza un modelo de procesamiento por tareas paralelas distribuidas. Cada tarea consume datos de flujos (por ejemplo, temas de Kafka), los transforma y produce nuevos flujos de salida. El marco maneja la gestión del estado, el paralelismo y la tolerancia a fallos mediante puntos de control (checkpoints) y almacenamiento local persistente, lo que facilita la recuperación tras errores sin pérdida de datos.



Apache Drill

Apache Drill es un motor de consultas SQL distribuido, de código abierto y basado en el paradigma schema-on-read, diseñado para explorar y analizar grandes volúmenes de datos estructurados, semiestructurados y, en algunos casos, no estructurados, sin requerir la definición previa de un esquema. Permite ejecutar consultas SQL de baja latencia sobre datos almacenados en múltiples formatos (como JSON, Parquet, CSV y Avro) y en diversas fuentes de datos, incluyendo sistemas de archivos distribuidos, bases de datos NoSQL y almacenamiento en la nube.



Arquitectura y funcionamiento

Apache Drill utiliza una arquitectura distribuida inspirada en el diseño de Google Dremel. No requiere un esquema fijo: interpreta la estructura de los datos en tiempo de consulta, lo que le permite trabajar con archivos semi-estructurados como JSON o Parquet, así como con datos almacenados en sistemas como Hadoop, Apache HBase o MongoDB.



Apache Presto

Apache Presto es un motor de consultas SQL distribuido de código abierto diseñado para realizar análisis interactivos sobre grandes volúmenes de datos. Permite ejecutar consultas rápidas a través de múltiples fuentes heterogéneas sin necesidad de mover los datos, lo que lo hace popular en entornos de análisis a gran escala.



Arquitectura y funcionamiento

Presto ejecuta consultas SQL de forma distribuida dividiéndolas en múltiples etapas y tareas que se procesan en paralelo sobre un clúster de nodos de trabajo (Workers), coordinados por un nodo Coordinator, responsable de analizar, optimizar y planificar la ejecución de las consultas. Su arquitectura utiliza un modelo de procesamiento en pipeline y prioriza la ejecución en memoria para reducir la latencia y ofrecer respuestas interactivas incluso sobre grandes volúmenes de datos.





Arquitectura y funcionamiento



Presto es un motor de consultas **SQL distribuido**, de código abierto, diseñado para ejecutar consultas analíticas interactivas de baja latencia sobre grandes volúmenes de datos desde múltiples fuentes.

¿CÓMO FUNCIONA?



1. ANÁLISIS Y PLANIFICACIÓN

El coordinador recibe la consulta SQL, la analiza, optimiza y genera un plan de ejecución distribuido dividido en etapas.



2. EJECUCIÓN PARALELA

El plan se divide en fragmentos (tareas) que se ejecutan en paralelo en múltiples nodos de trabajo del clúster.



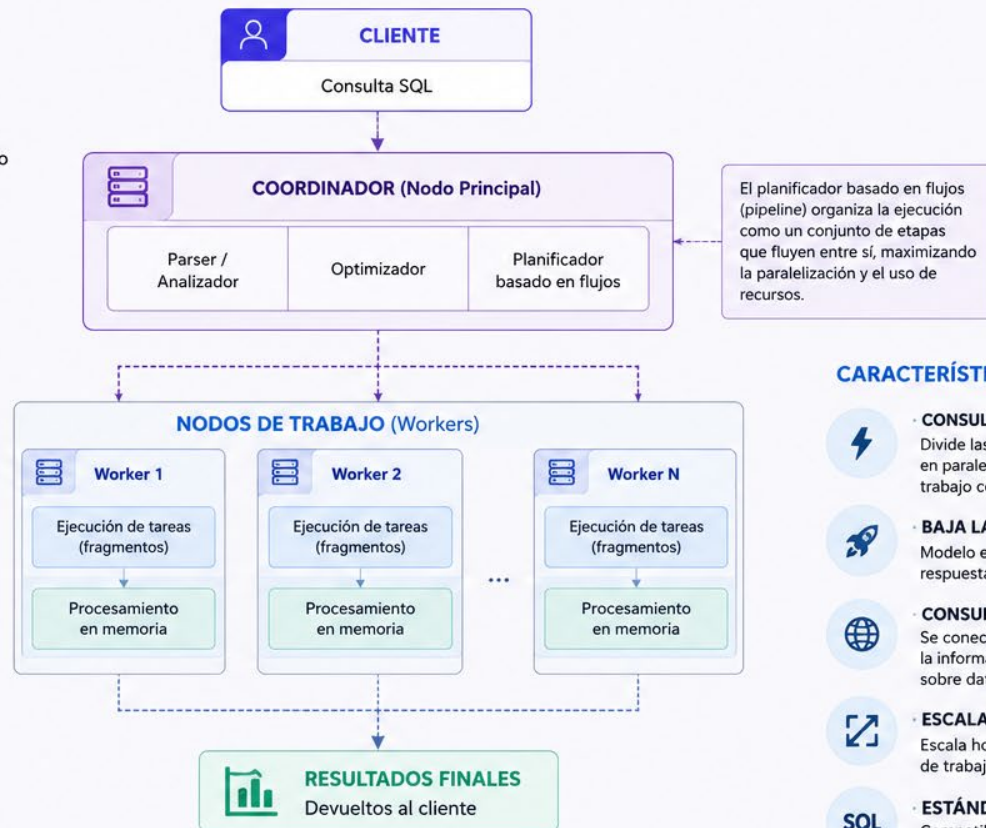
3. MODELO EN MEMORIA

Los datos se procesan en memoria siempre que es posible, reduciendo E/S a disco y minimizando la latencia.



4. AGREGACIÓN Y RESULTADOS

Los resultados parciales se agregan y envían al coordinador, que devuelve el resultado final al cliente.



CARACTERÍSTICAS CLAVE



CONSULTAS DISTRIBUIDAS Y PARALELAS

Divide las consultas en fragmentos que se ejecutan en paralelo a través de un clúster de nodos de trabajo coordinados por un nodo principal.



BAJA LATENCIA

Modelo en memoria y ejecución vectorizada para respuestas rápidas en cargas analíticas interactivas.



CONSULTAS FEDERADAS

Se conecta a múltiples fuentes de datos sin mover la información, permitiendo consultas unificadas sobre datos heterogéneos.



ESCALABLE Y ELÁSTICO

Escala horizontalmente agregando más nodos de trabajo según la demanda.



ESTÁNDAR ANSI SQL

Compatible con SQL estándar para facilitar su uso e integración.

FUENTES DE DATOS COMPATIBLES (Consultas Federadas)



Apache Hive



Amazon S3



Bases de datos relacionales
(MySQL, PostgreSQL, SQL Server, Oracle, etc.)



Sistemas NoSQL
(MongoDB, Cassandra, HBase, Redis, etc.)



Elasticsearch



Kafka



Y muchas más...

presto

Consulta rápida. Datos donde están.
Escalabilidad sin límites.

<https://prestodb.io>

Apache Calcite

Apache Calcite es un framework de código abierto para el procesamiento y optimización de consultas SQL, desarrollado por la Apache Software Foundation. Proporciona capacidades avanzadas de análisis sintáctico, validación semántica, optimización basada en reglas y costos, planificación de consultas y federación de datos, sin incorporar un motor de almacenamiento propio.



Arquitectura y funcionamiento

Apache Calcite – Descripción fundamentada

Apache Calcite es un marco de gestión de datos de código abierto que implementa un motor de optimización y ejecución de consultas SQL independiente del almacenamiento. Proporciona un **analizador SQL completo, validación semántica, transformación y optimización de consultas basadas en un modelo relacional extensible**.

No almacena datos ni ejecuta E/S de forma nativa; en su lugar, se conecta a múltiples fuentes de datos a través de **adaptadores (connectors)** y delega la ejecución final a sistemas externos.

Desarrollado por la Apache Software Foundation, se utiliza ampliamente como **capa de optimización** en sistemas distribuidos, motores de bases de datos, plataformas de streaming, lagos de datos y servicios de datos empresariales.

Su diseño **modular y extensible** permite personalizar y ampliar reglas de optimización, tipos de datos, funciones, operadores y conectores para adaptarse a diversos dominios y fuentes.

Fundamentos que sustentan la afirmación



Sin almacenamiento propio

Calcite no almacena datos ni gestiona archivos. Opera sobre interfaces abstractas y delega la lectura/escritura a sistemas externos vía conectores.



Análisis, validación y optimización SQL

Incluye parser SQL, validador basado en metadatos, reescritura de consultas y optimizador basado en reglas (volcano/CBO).



Diseño modular y extensible

Componentes intercambiables: reglas, convenciones, tipos de datos, funciones, operadores y conectores (Schema/Adapter).



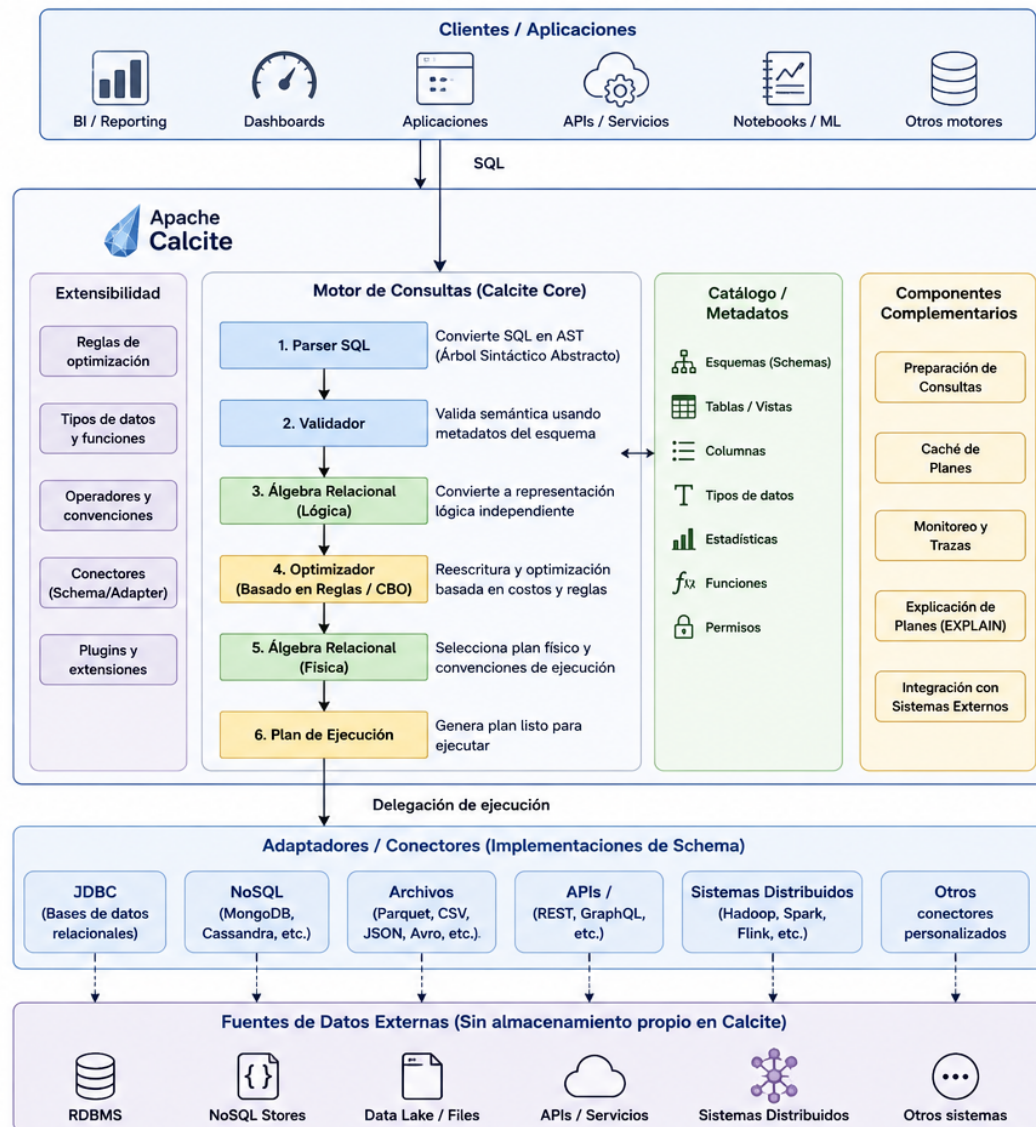
Integración en ecosistemas modernos

Usado en múltiples proyectos y motores como capa de optimización: sistemas distribuidos, motores embebidos, plataformas de datos y API de servicios.

Fuentes

- Documentación oficial de Apache Calcite: <https://calcite.apache.org/docs/overview.html>
- Whitepaper: "Apache Calcite: A Foundational Framework for Optimized Query Processing" (ApacheCon EU 2017)
- Proyecto Apache Calcite en GitHub: <https://github.com/apache/calcite>

Arquitectura de Apache Calcite



Apache SAMOA

Apache SAMOA (Scalable Advanced Massive Online Analysis) es un framework de código abierto diseñado para desarrollar y ejecutar algoritmos de aprendizaje automático (Machine Learning) y minería de datos sobre flujos continuos (data streams) en entornos de procesamiento distribuido.

Su arquitectura proporciona una capa de abstracción que desacopla la implementación de los algoritmos del motor de procesamiento subyacente, permitiendo su ejecución sobre plataformas como Apache Storm, Apache Flink y Apache Samza, entre otras.

Apache SAMOA
Scalable Advanced Massive Online Analysis

Arquitectura y funcionamiento

Apache SAMOA: Marco para Aprendizaje Automático en Flujos a Escala

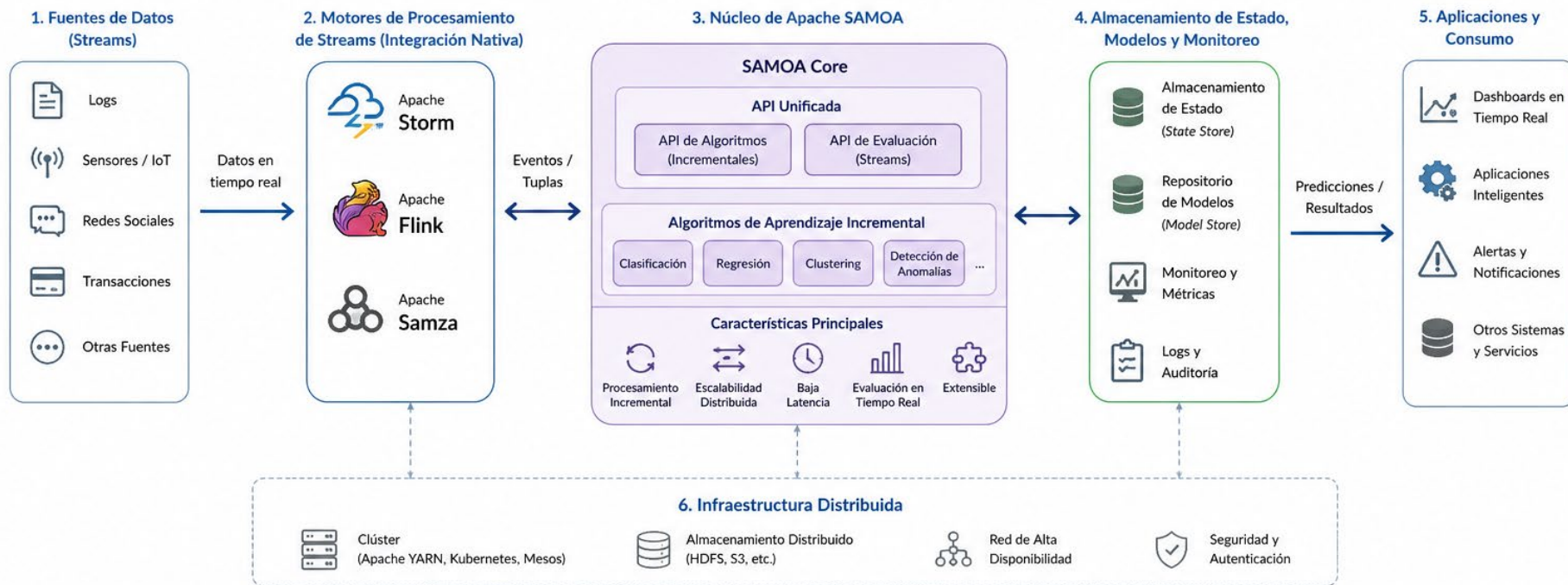
Apache SAMOA (Scalable Advanced Massive Online Analysis) es un marco de software de código abierto para aprendizaje automático y minería de datos sobre flujos (*streams*) en entornos distribuidos.

- ✓ Procesa datos en tiempo real y de forma incremental, actualizando los modelos conforme llegan nuevos datos.
- ✓ Diseñado para escalar horizontalmente y operar de manera distribuida en clústeres.
- ✓ Proporciona una API unificada para algoritmos de aprendizaje incremental y evaluadores de flujos.
- ✓ Se integra nativamente con motores de procesamiento de streams como Apache Storm, Apache Flink y Apache Samza.
- ✓ Permite construir aplicaciones de analítica en tiempo real robustas y de baja latencia.



✓ **Corroboración:** La información anterior es correcta. Está respaldada por la documentación oficial de Apache SAMOA y su integración con motores de *streams*. SAMOA fue originalmente un subproyecto de Apache Mahout y luego se independizó como proyecto de nivel superior en la Apache Software Foundation.

Arquitectura de Apache SAMOA





Actividad de Aprendizaje

Fase 1: Autoaprendizaje (Investigación individual)

1. ¿Cuáles son las diferencias conceptuales entre el procesamiento batch y streaming, y en qué casos se recomienda cada uno dentro de entornos Big Data?
2. Investiga cómo los frameworks como Hadoop y Spark gestionan la tolerancia a fallos y compáralos con otros sistemas distribuidos modernos.
3. ¿Qué papel cumplen los sistemas de mensajería como Kafka en arquitecturas de datos modernas (Data Pipelines) y cómo se integran con herramientas de análisis?
4. Explica el concepto de procesamiento en memoria y analiza por qué representa una ventaja significativa en frameworks como Spark frente a Hadoop MapReduce.
5. ¿Cómo contribuyen los frameworks de consultas distribuidas (como Presto o Drill) a la toma de decisiones en tiempo real en empresas?

Fase 2: Discusión (Debate crítico grupal)

1. ¿Es Hadoop todavía relevante frente a tecnologías más modernas como Spark o Flink? Argumenten a favor y en contra.
2. ¿Qué framework consideran más adecuado para aplicaciones en tiempo real: Kafka, Flink o Storm? Justifiquen su postura con ejemplos.
3. ¿Es mejor centralizar el procesamiento de datos en un solo framework o utilizar un ecosistema de múltiples herramientas? Debatan ventajas y desventajas.
4. ¿Hasta qué punto la complejidad de estos frameworks limita su adopción en pequeñas y medianas empresas?
5. ¿La integración de minería de datos con bases de datos multimedia realmente aporta valor en todos los contextos, o solo en industrias específicas?

Fase 3: Reflexión (Síntesis grupal)

1. ¿Qué postura final adoptó el grupo respecto al uso de múltiples frameworks en Big Data y por qué?
2. ¿Qué aprendizajes nuevos surgieron durante el debate que no habían considerado en la fase de investigación?
3. ¿Cómo cambió su percepción sobre el rol de los frameworks en la minería de datos tras la discusión?
4. ¿Qué dificultades encontraron al analizar y comparar los diferentes frameworks?
5. ¿Cómo aplicarían lo aprendido en un contexto real (empresa, proyecto o investigación)?



Universidad Nacional de Trujillo



Cierre



Gracias



Universidad Nacional de Trujillo